

Cloud Computing and Software Engineering Spring 2025 - Report

Anastasja (Nathaniel) Helgesen

June 2026

1 Introduction

My primary research domain is tournament theory as applied to multi-class classification. I have found the mathematics of tournament theory, and its link to simplex geometry, to provide useful lenses into the outputs of both classical and quantum classification models. The basic premise is to extend the classification problem from a standard one-vs-all approach to a one-vs-one problem, allowing the decision for a class to be made in “edge space” rather than “vertex space.” In quantum models this functions as a form of error correction, enabling scalable robustness against noise from model error, hardware error, and quantum stochasticity. In classical models this enables us to train established classification models to output full tree-linked hierarchies, especially when treating the output as a mass distribution rather than a probability (leading back to geometry). In short, my research explores the intersection of mathematics, machine learning, and quantum computing, with a focus on information representation, geometric constraints, and sensible evaluation structures. Rather than developing new models like in most machine learning research, my work imposes structure on existing models through methods inspired by back-burner mathematical concepts.

This perspective differs from much of what I see in AI engineering, where the focus is placed on developing, deploying, and maintaining model-centric systems. Treating this as gospel, my research appears somewhat disconnected from software engineering practice. As a result, I will focus the thesis of this article on how current AI engineering practices tend to treat models and pipelines as the primary units of design, while structural aspects such as constraints, representations, and decision consistency remain underdeveloped.

2 Lecture Principals

2.1 Science vs Engineering + ML Engineering

The most abundant difference between my work and engineering stems exactly from the difference between science and engineering given in the slides. More or less, science explains, “this is what works, when it works reliably, and what it implies,” whereas engineering explains, “which outside forces will try their hardest to break this, and what leaks, bugs, damage, hardware constraints, and customer satisfaction can we plan for, avoid, or otherwise build in imperviousness for.” Given my work sits in the space of measurement, evaluation, and constraint, I would argue that it is primarily a scientific endeavor—or at the very least, a new lens for engineers to use when designing systems. This of course becomes more complicated when we consider that machine learning as a field tends to cross the engineering and science fields at its core. I might argue that this viewpoint primarily describes the aforementioned “standard practice” in ML, which is to design better and better models or frameworks to transmute information accurately from one form to another. My work certainly lives *around* that subtext, but then it begs the question—does science primarily focused on regrounding the work of ML researchers in better mathematical frameworks still count as engineering if I am not actually producing deployable code/models/the like?

When looking at the experiment → deployment → observation → refinement loop, the cleanest place to put this sort of work on metrics, representation, and constraints is either experimentation or observation. As none of my code itself is meant to be necessarily deployed, it avoids the need to be “engineered,” but rather “engineered *with*.” By similar logic, my work sits somewhere in the first two stages of the ML-TRL ladder, being theory that thus influences future engineering. I might even go so far as to say that my work hearkens back to older fields where scientists established rules

and guidelines, and engineers just used those principals—unlike ML where the perspective is largely and arguably toxically “deliverable-driven.”

2.2 Behavioral Software Engineering

This set of slides raised a few important connections to my work and the effect human behavior has on its usage and acceptance. Since humans rely on heuristics, one who designs better heuristics and models of success might think their work quite useful, but a very strong bias that humans seem to have is to favor whatever heuristics they are already used to—even if those heuristics are shown to provide less insight, guidance, or direction. I of course aim to make what little code I make simple to plug in so engineers can validate the usefulness of my new tools, but as they cannot be found in textbooks, many might view them as redundant or useless. In short, there is a very serious concern that availability and confirmation biases may make my work less immediately successful, as new or extra tools that are hard to find can be just inconvenient enough for people to look past them.

I find when I give presentations on my work that what I feel is simple, elegant, and intuitive requires many more rounds of questions to clarify due to the perpendicular view from the standard metrics and representations my methods are derived from and useful for. That is to say, my work aims to align with human thinking as best as possible, but due to existing mental frameworks that work often “well enough,” my work is different enough to require more up front effort to align with ML modernity. In short, the cognitive load of learning my methods may pose a serious obstacle to their uptake by engineers in the field of ML engineering.

3 Link to Guest Lectures

The lecture by Julian Frattini contained many useful engineering insights that lend hope to the issues I have already pointed out in making my work useful to engineers. The biggest connection is simply to his statement that we should “understand the problem before [we] build the solution.” It seems redundant to say again that my work is more focused on the problem space than on actually solving anything, but from Frattini’s perspective, there seems to be a definition of my work that aligns with “defining the problem.” For example, wanting a system that classifies wild animals in a nature cam, defining the problem as picking the “best” option can lead to solutions that, when wrong, are very wrong. Defining the problem instead as “output accurate likelihoods/ranking of every possible animal,” then the model one designs may require extra help when deciding between likely options, but at least the model won’t classify a rat as an elephant.

I also appreciate Frattini’s discussion of “levels of abstraction” in that it helps define what I have otherwise considered a problem as a novel layer in a more common abstraction schema. I would say my work sits at an intermediate abstraction layer, since it helps lock into the “what”, but not necessarily the “how.” This becomes pedantic if we consider “how to measure” as a form of “how,” though I am quite confident the “how” Frattini refers to as “how to solve the problem at hand,” and this problem is rarely “how to measure the degeneracy of a model trained for a specific task without measuring that task explicitly.” My work does not define concrete system or model decisions as much as it explores mathematical forms that can be used as more insightful representations of information learned by models. From a systems perspective, it is as simple as working these frameworks in as part of the system, and from a model perspective, the frameworks are agnostic. Any model designed to transform an input to a series of classes can leverage the framework as an additional probe. This reads to me as something in between the two, while regretfully throwing out a personal need to learn all there is about either.

4 Book Chapters

The book chapters dig into this separation even further. The introduction redefines the same concepts as are used in the lecture slides, defining software people as focusing on systems, and data scientists as focusing on models. From this perspective, my model sits a bit closer to data science, as my work functions as better lenses into data than simple single-class classification. I still feel as though there is a slight over-separation here though. I do not need to know full model structure, full problem structure, or deployment ease when I build my mathematical theories that guarantee more structured multi-class classification. As mentioned before, my work is data agnostic aside from requiring the problem be multi-class classification. What hierarchical ranks exist, how they are useful, and what such an ordered list of classes may provide are not questions I either ask or answer. From

this perspective, I have a unique take on the distinction as incomplete. I doubt anyone would argue I work somewhere in the realm of "building and integrating systems to solve a unique challenge," but I don't accomplish this by being a master of either subdomain defined in Chapter 1.

Chapter 2 pushes the argument that these two types of engineers should learn aspects of each others' domains in order to build more coherent systems which are faithful to the data but still provide a benefit to a broader goal. For example, maybe classifying the wild animals is meant to be a useful tool in a larger system tracking the movement of endangered species. The model being more accurate does not necessarily mean it is more useful if when it is wrong it is extremely wrong or if it takes a very long time to process the information. I have personally seen this in Africa, where tracking the movement of wild elephants was a two step process—capture videos onto a flash drive, then process them in the US after a long flight. This may not be useful. Looping in wildlife experts who know about the environment can help improve the model, and looping in software engineers who understand the cloud, the hardware limits, and the tradeoffs can possibly make the model run more locally. Expecting anyone to know all of these aspects is ludicrous.

To this end, I do not believe fully merging is a reasonable goal by any means. My experience in game design has told me that programmers do not need to know how to 3D model, they just need to know what "API-like" entry-points there are to the artists' models (visual ones, this time, not ML) and how to interact with them. Likewise, the modelers may benefit from knowing what sorts of conveniences they can leave in for the programmers. This same "API-like" knowledge is extremely common in literally every field of everything, and it seems quite obvious this type of thinking is equally useful in ML deployment. Data scientist says, "put image in, get guess out," then the software devs can just use the model like any other function—ad nauseum as models become more complicated.

5 Paper Summaries

5.1 RAGProbe: Breaking RAG Pipelines with Evaluation Scenarios

This section will summarize and connect to the paper sharing a name with the subsection title, Sivasothy et al. [2024].

5.1.1 Core ideas and their SE importance

The main premise and key arguments in this paper start from the seemingly established problem that RAG models are hard to evaluate. This stems from the fact that RAG models augment LLMs which have their own limitations, and citing exact information may not be a primary training goal for such LLMs. This leads to models which confidently use outside knowledge to answer questions from sources incorrectly. In proper software engineering fashion the authors wanted to build a universal test protocol for RAG systems that tests for how well they can answer questions from their own corpora, making for flexible and unbiased comparison, testing, and deployment. Their protocol creates both the questions and the answers using different combinations of corpora, RAG, and LLM which are constraint and measured via 6 question types. This is likely a wonderful tool for chat-bots trained on a company's intranet, better research assistant chat-bots, or the like, by giving an SE-like tool that front end developers (not data scientists) can use to assess available tools when designing their products.

5.1.2 Relation to my work

I find this paper to be similar to my work in the sense that both focus on systematic evaluation of models designed by others. I really appreciate the authors' adversarial approach to testing, exposing failure rather than success, as this seems a bit more rare in the data scientist domain.

5.1.3 Larger AI-intensive project

To stick to an established domain, I will apply such thinking to my hypothetical trap-camera project. I could see a similar test suite that is designed to find which animals are often misclassified, what they are often misclassified as, and compile the results across many different models trained on different data. Such a test could help different teams working in different ecosystems figure out which models are best for their specific environment. As an enhancement, such a test could be designed to probe more specifically at endangered animals or other targets of interest. Though what I describe is a simple system, I would stress that RAGProbe's specific strength and addition here

is that the way they generate failure cases is based not only on the model but also on the varying datasets. In the wild, no plot of land is likely to contain the same types of creatures, meaning a lot of flexibility and bulk stress-testing can help find not only model weaknesses but data weaknesses as well. Such a system drives directly into my research, as I work directly with multi-class confusion and rank-similarity, meaning my metrics could be used to assess such failures.

5.1.4 Adaptation of my research

I chose this paper because to be very honest, it feels very similar to what I work with as is. Hierarchical multi-class classification (HMCC) is already an extension of a task that was deemed some combination of *too easy* and *weird failure cases*. As mentioned in other sections, accuracy stops at “did the model predict the correct class by way of highest activation,” whereas in HMCC we have a ground truth tree connecting every class together, enabling us to more granularly measure a model’s full understanding of the full class list. This comes from asking the question, is classifying a picture of a corgie as a shiba inu really *equally bad* to classifying that same image as a submarine? Accuracy might say so quantitatively, but this is intuitively very incorrect.

From this starting point, I draw the same connection I did with the camera-trap example. My metric functions as a probe into classification models and different datasets with hierarchies. This ground truth can be genus/species/so on, vehicle make/model, or any other arbitrary clustering. With this signal, we have our “failure cases,” similar to RAGProbe, and can use that to define acceptable failure thresholds. My metric already measures the earth mover difference between a predicted rank tree and ground truth one, but maybe some degree of failure is accepted. One dataset I often work with is FGVC aircraft, which is a small dataset of images of planes with a ground truth mapping the make, model, and sub-model (versions of a similar plane etc)—which may be entirely unnecessary in a hypothetical target tracking algorithm, for example. In this case, failure may actually mean a model doesn’t classify the make or model wrong, but is still useably correct if it guesses Boeing 737-A instead of Boeing 737-B, or whatever.

At this point, we have established flexibility in dataset, model, and failure case. Beyond that, my metric already probes for more than accuracy ever could, and provides much more insight into how a model is performing as well. What is missing from my work that can be inspired by RAGProbe is the very same as what I highlighted earlier. There would need to be some method of plugging in a model and dataset and generating failure cases on the fly. This does not seem entirely impossible from a creativity standpoint, but the struggle will lie in the nature of the ground truth. An established hierarchy of class labels is not an easy thing to acquire, and having it basically defines a quadratic number of possible failure conditions. My metric sweeps over this by treating each failure equally and conglomerating them into a single “failure score,” of sorts. Having already mentioned that designing dataset specific probes may be easy to implement for domain experts to probe their specific failure cases, but there is another option I hope to eventually try that can make my metric more agnostic.

There may be some way to establish a global predicted hierarchy tree from a model and determine if the tree gives rise to anything useful. While I am unsure how this work (otherwise I would have published it and be quite famous), the idea would be to use aspects of my geometric work to assess the quality of a predicted tree. If a model clusters cats, dogs, and foxes into a single rank level, and that level is a sublevel of *animals*, we as humans can agree this a “reasonable,” tree. If another model clusters sailboats, dogs, and ice cream into one tree, we can reasonably assume it not capturing global similarity and say it “failed worse.” Establishing this test suite is one of my major goals in my research, as I would like my work to eventually help humans classify difficult species or more genetically establish taxonomies to more rigorously define *similarity* in plants and animals.

All this to say, the strongest idea from this paper that can help me propel my research is the notion of establishing both the questions and the answers from the data. There may not be a neat parallel between RAG, which can be called *correct* or *incorrect*, and my method which has a quadratic number of *failures* which are more softly compared and ranked. It is an extremely interesting thing to think about though. I am excited to see what kind of data-defined failure cases can be created to help assess HMCC models more thoroughly—or have I already answered that? Impossible to say for now.

5.2 How Do Model Export Formats Impact the Development of ML-Enabled Systems?

This section will summarize and connect to the paper sharing a name with the subsection title, [Parida et al., 2025].

5.2.1 Core ideas and their SE importance

The basis of this paper is very simple—ML models are used in very different ways and environments than those they are trained in. Training models is often done in Python environments optimized for CUDA integration and light compilation. Deployment however, is done by compressing and encoding the parameters, layers, and other model components into one of several data types designed for the purpose of integrating “completed” models into full stack projects. The most common (and maybe only) resources for this have pros and cons, and some work better in different situations than others. The authors asked the question, “what resources, experiences, and opinions do the people who work with ML models have in regard to deployment?” They give detailed breakdowns of each data type from the perspective of devs, data scientists, and ML practitioners, using surveys and hands on integration debriefs.

5.2.2 Relation to my work

In truth, this project is not directly related to my work, but that is what made me want to look at it. I knew about different model deployment data types, but I have never actually integrated any of my ML work with any full stack projects. This is certainly true relative to the type of complexity these formats are accustomed to—that is, I have never made a full LLM chatbot or target tracking system that runs in browser, which seem to be some of the capabilities such data types have. Now I can ask questions in the vein of “how well will this work in an ONNX file (one such data type)?” about the layers and matrices my work introduces.

5.2.3 Larger AI-intensive project

Sticking with the camera trap example, this paper seems very relevant. Researchers capturing wildlife data create large datasets; experts label the data; data scientists train models to track animals; my metrics assess the failure types; now a model for detecting endangered species needs to be deployed. According to the authors, the easier way to do this is almost always ONNX due to its multi platform flexibility and significantly smaller size than the easier TensorFlow SavedModels which contain model details for black box application. This ease gained by Tensorflow is easy enough to overcome if the models are defined in separate smaller code.

Once a model trained on camera trap data is deployed with ONNX, it can be integrated easily into tracking projects, conservation projects, or hobby animal recognition apps both in Java and Python (making up a humongous proportion of web development code). Since ONNX models are so small, such an app could even run on a smartphone, model dependent.

5.2.4 Adaptation of my research

One thing I have started thinking about more and more as I progress in my ML PhD journey is how annoyingly many models can vary from one another, and how this always turns into a very messy spiderweb of ablation, loss, model, dataset, preprocessing, etc. I recently moved to defining models, functions, trainers, and anything else that may vary in an ML pipeline in a singleton `context` dictionary. This allows for building simple `yaml` files that allow for modification of default values, simpler function webbing (or whatever this is really called—functions that call functions with varying defaults and call structures), and concrete repeatable model definitions. The authors have shown pretty clearly that `yaml` is a bad way to save full models, which makes sense, but this paper has me thinking about how such `yaml` context configurators could work in tandem with the well optimized ONNX datatype.

Since the biggest downside to ONNX is essentially that the model layers are not outright defined, one must either use a specific tool to extract the information needed to build the model, or start with an established codebase and load the ONNX parameters into that. This process seems rather similar to how we store parameters as checkpoints in the training procedure, but is likely much faster to read and load (I have never used any of the deployment data types). If the biggest issue to running an ONNX model is a lack of model details, why not store model definitions separately? The idea I have in mind is a sort of model ecosystem that uses my `context` datatype by default, and stores optional tweaks to training and inference values or layer alternatives etc in `yaml` format.

Being consistent with this context call signature, defaulting values for “missing” context variables, and saving the original python model definitions would enable ONNX to overcome this simple issue. All that said, this may not be the best way to think about how these deployment data formats are expected to work. If I know ahead of time that the person deploying a model is using PyTorch, then

TorchScript is easiest, as all the math and layer integrations are part of that format. It simply is harder to integrate TorchScript with Javascript, which may sometimes be a problem. I do think, in general, this paper has me thinking about possible upsides to this `yaml/context` framework I have been using, in hopes that it not only helps scientific iteration, but also full stack integration.

A more important takeaway might be in considering how my additional model artifacts (layers, metrics, matrices) work in conjunction with ONNX. I am confident that TorchScript could make my stuff work, but since many of my layers use linear algebra libraries and are highly custom, there may not be a direct route into integrating them with ONNX. For all I know, I would have to write additional ONNX code in order that someone could plug and play with my additional layers as such. Perhaps the best solution is to write a TensorFlow and a JAX version of my gizmos, such that any of the studied tools could be used by other data scientists who have their own favourite pipelines. The only risky downside to my method is the requirement of more output neurons from a standard model, but that isn't a formatting issue insomuch as it is a scaling tradeoff. All in all, the layers and mathematics that make my methods, functions, losses, and layers work all require no additional learned parameters within themselves. There is one very interesting scientific question with my Bradley-Terry method, however, as it uses a quite large but precomputable adjacency matrix. I am unsure if such a matrix needs to be saved or just left as part of the module definition, which makes me very confused about these deployment data formats. I finish with an unanswered question. Are these deployed models expected to be a self contained black box million parameter function, or is there some level of acceptance that the architecture, parameters, and settings are truly three separate objects? If the latter is the case, I see no reason to store large "default setting" matrices in the deployment file, but if so, then there may be future drama from the potential scale of that matrix.

5.3 Research Ethics and Sythesis Reflections

To find the papers, I searched the CAIN 2025 proceedings and screened titles and abstracts for terms connected to classification, evaluation, testing, and failure discovery. I had hoped to find papers discussing multi-class classification, but most seemed to be more system-level AI engineering stuff rather than classification theory. To circumvent this, the first paper was chosen from the broader category of "automated evaluation of AI system behavior," as this is closer to my movement from top-1 accuracy to full ordering of classification outputs. I chose the second paper because integration of ML artifacts into larger software systems is something I have consciously avoided in my research (I like math). This thusly became the first time I have considered connecting my work to the higher level software engineering like testing, interfaces, and system integration. Given the paper's broad applicability, it seemed a safe entryway into this line of thinking.

References

- Shreyas Kumar Parida, Ilias Gerostathopoulos, and Justus Bogner. How do model export formats impact the development of ml-enabled systems? a case study on model integration. In *2025 IEEE/ACM 4th International Conference on AI Engineering – Software Engineering for AI (CAIN)*, page 48–59. IEEE, April 2025. doi: 10.1109/cain66642.2025.00014. URL <http://dx.doi.org/10.1109/CAIN66642.2025.00014>.
- Shangeetha Sivasothy, Scott Barnett, Stefanus Kurniawan, Zafaryab Rasool, and Rajesh Vasa. Rag-probe: An automated approach for evaluating rag applications. *International Conference on AI Engineering – Software Engineering for AI*, 2024. URL <https://arxiv.org/abs/2409.19019>.